National University of Computer and Emerging Sciences



LABORATORY MANUAL

for

Data Warehousing & Business Intelligence Lab

(DL3004)

LAB 06 • Dimensional Modeling II & Physical DW Implementation

Course Instructor	Mr. Muhammad Ishaq Raza
Lab Instructor	Mr. Shahzeb Mubashar
Semester	Fall 2026
Program	BS (Data Science)

FAST School of Computing
FAST-NU, Lahore, Pakistan

Lab Manual 06

Objectives

By the end of this lab, students should be able to:

- Create separate staging and data-warehouse schemas in PostgreSQL.
- Load operational source extracts into staging tables without prematurely imposing warehouse keys.
- Profile and standardize staging data before dimensional loading.
- Implement dimensions with warehouse surrogate keys while retaining source business keys.
- Generate and load a reusable date dimension.
- Load dimensions before facts and map source business keys to surrogate keys.
- Enforce primary-key and foreign-key constraints in a complete star schema.
- Detect duplicate business keys, unmapped dimension references, and row-count/revenue reconciliation errors.
- Validate that the physical warehouse preserves the intended grain and analytical totals.

Tools	PostgreSQL, pgAdmin 4, diagrams.net / dbdiagram.io
Datasets	Lab 05/06 customer, product, region, order and order-item source CSV files
CLO Mapping	CLO 2, CLO 3
Submission	SQL file + physical star-schema diagram + pipeline diagram + validation screenshots + evidence document

PRE-LAB: Requirement

Pre-Lab Requirement
Complete the Lab 05 sales dimensional design and download the Lab 05/06 dataset pack. Students should have PostgreSQL and pgAdmin 4 working. Lab 06 implements a warehouse from the same operational extracts used in Lab 05.

1. Business Scenario

The dimensional design from Lab 05 has been approved. CampusCart now needs a physical PostgreSQL warehouse that separates raw ingestion from curated dimensional storage. Source systems identify customers and products using business codes, but the warehouse must assign its own surrogate keys and store those keys in the fact table.

You will implement a complete load path: Source CSV → Staging → Cleaned/validated dimension rows → Surrogate-key dimensions → Fact mapping → Reconciliation.

IMPLEMENTATION PRINCIPLE: Dimensions before facts

A fact row cannot safely store warehouse foreign keys until the relevant dimension rows exist. Load and validate dimensions first, then map each source business key to its warehouse surrogate key.

2. Supplied Source Extracts

File	Source Grain	Warehouse Use
lab05_06_source_customers.csv	One source customer record; contains a few duplicates/text inconsistencies	DimCustomer
lab05_06_source_products.csv	One source product record; contains a few duplicates/text inconsistencies	DimProduct
lab05_06_source_regions.csv	One region	DimRegion
lab05_06_source_orders.csv	One order header	FactSales context + DimChannel
lab05_06_source_order_items.csv	One order line	FactSales measures

The Date dimension will be generated in PostgreSQL rather than supplied as a CSV.

3. Create the Physical Architecture

Task 3.1 — Create Schemas

Create separate schemas:

```
CREATE SCHEMA IF NOT EXISTS lab06_stg;
CREATE SCHEMA IF NOT EXISTS lab06_dw;
```

Task 3.2 — Architecture Diagram

Draw a small pipeline diagram showing Source CSVs → lab06_stg → lab06_dw dimensions → lab06_dw.fact_sales. Label where data-quality checks and surrogate-key mapping occur.

4. Build and Load Staging Tables

Task 4.1 — Create Staging Tables

Create one staging table for each supplied CSV. Staging should preserve source business keys and use permissive but sensible types. Do not add warehouse surrogate keys to staging.

Task 4.2 — Import CSV Files

Use pgAdmin Import/Export Data to load all five source extracts. Record row counts immediately after loading.

Task 4.3 — Profile Data Quality

Write reproducible queries that identify:

- Duplicate customer_code values.
- Duplicate product_code values.
- Leading/trailing whitespace in descriptive text.
- Inconsistent casing in customer_segment, category, or brand.
- NULLs in required business keys.
- Order items whose order_id does not exist in staging orders.

CHECKPOINT: Do not hide source defects

Your staging layer should preserve what arrived. Clean or deduplicate when loading curated dimensions, but keep staging useful for audit and reconciliation.

5. Implement Warehouse Dimensions

Task 5.1 — Create DimRegion, DimCustomer, DimProduct and DimChannel

Each dimension must contain a surrogate primary key generated by PostgreSQL and must retain the corresponding business/natural key where applicable. Add a UNIQUE constraint on cleaned business keys in the warehouse dimension.

Dimension	Surrogate PK	Business Key / Identifier
dim_region	region_key	region_code
dim_customer	customer_key	customer_code
dim_product	product_key	product_code
dim_channel	channel_key	channel_name

Task 5.2 — Standardize While Loading

Load cleaned dimensions from staging. Use appropriate transformations such as TRIM and consistent casing. Resolve exact duplicate business-key rows so each warehouse business key maps to one surrogate key.

Task 5.3 — Validate Dimension Uniqueness

For every dimension, compare total rows with distinct business keys and prove the warehouse dimension contains one row per business key.

6. Build the Date Dimension

Task 6.1 — Create DimDate

Create lab06_dw.dim_date with date_key as an integer in YYYYMMDD form plus full_date, day, month, month_name, quarter, year, and day_of_week. Use date_key as the primary key.

Task 6.2 — Populate the Required Range

Generate rows covering at least 2024-01-01 through 2026-12-31 using PostgreSQL generate_series(). Do not manually insert dates.

Task 6.3 — Validate Date Coverage

Confirm that the minimum and maximum order_date in staging both map to DimDate and that each full_date is unique.

7. Create and Load FactSales

Task 7.1 — Define the Physical Fact Table

Create lab06_dw.fact_sales at the Lab 05 grain: one row per source order line. Include line_id as the fact primary key, order_id as a degenerate identifier, dimension surrogate foreign keys, and the required numeric measures.

Required foreign keys: date_key, customer_key, product_key, region_key, channel_key.

Required measures: quantity, unit_price_pkr, discount_pct, gross_revenue_pkr, discount_amount_pkr, net_revenue_pkr.

Task 7.2 — Surrogate-Key Mapping

Populate fact_sales by joining staging order items to staging orders and then mapping source business keys to warehouse dimensions. The final fact INSERT must store surrogate keys, not customer_code/product_code/region_code.

Task 7.3 — Explain the Mapping

For one loaded fact row, show the original customer_code and product_code alongside the resolved customer_key and product_key. Explain why the warehouse fact prefers the surrogate keys.

8. Data-Quality and Integrity Validation

Task 8.1 — Orphan Checks

Write orphan-detection queries for every dimension foreign key in fact_sales. The final warehouse should contain zero orphan facts.

Task 8.2 — Grain Reconciliation

Compare the number of source order-item rows with fact_sales rows. Explain any difference. Under the supplied clean transactional keys, the final counts should reconcile exactly.

Task 8.3 — Measure Reconciliation

Compare SUM(quantity), SUM(gross_revenue_pkr), SUM(discount_amount_pkr), and SUM(net_revenue_pkr) between staging order items and fact_sales. Differences indicate a mapping or duplication defect.

Task 8.4 — Constraint Test

Without permanently changing the dataset, demonstrate one INSERT that should fail because of a foreign-key or uniqueness constraint. Capture the error and explain which integrity rule protected the warehouse.

9. Analytical Validation of the Star

Task 9.1 — First Warehouse Query

Using only lab06_dw tables, report 2026 net revenue by region and product category, highest revenue first.

Task 9.2 — Multi-Dimensional Query

Report quarter, customer segment, channel, total net revenue, total quantity, and distinct orders for 2026. Use DimDate attributes rather than parsing date_key text.

Task 9.3 — Compare Source vs Warehouse Result

Answer one identical revenue question once from staging/source tables and once from the dimensional warehouse. The totals should agree. Compare readability and number of joins.

10. Load-Failure Debugging Challenge

A developer reports that the warehouse has more fact rows and more revenue than the source. Investigate the following possible causes and write a diagnostic SQL query for each:

- A dimension contains duplicate business keys, causing a many-to-many join during surrogate mapping.
- The fact-loading query joins to orders twice or at the wrong grain.
- A source order line was loaded more than once.
- A business key was cleaned differently in the fact mapping than in the dimension load.

Task 10.1 — Root-Cause Rule

Write a short rule explaining why row-count reconciliation alone is insufficient and why measure reconciliation must also be performed.

11. Applied Implementation Challenge

Challenge	Required Evidence
A — Surrogate Mapping Trace	Choose three source order lines and show source business keys → warehouse surrogate keys → final fact row.
B — Warehouse Integrity	Produce a compact validation result showing dimension uniqueness, zero fact orphans, matching fact row count, and matching net revenue.
C — Broken Load Simulation	Create a temporary copy or CTE that intentionally duplicates one dimension mapping; demonstrate how fact row count/revenue can inflate.
D — Physical Design Decision	Explain why staging should retain raw business keys while the curated fact stores surrogate keys.

12. Submission Requirements

- Lab06_<RollNo>.sql containing schemas, staging DDL, imports/validation queries, dimension DDL/load, Date dimension generation, fact load, reconciliation, and analytical validation.
- One physical star-schema diagram with surrogate PKs, business keys, FKs, measures, and fact grain.
- One small staging-to-DW pipeline diagram.
- Screenshots showing dimension mappings, zero-orphan validation, row-count reconciliation, and measure reconciliation.
- A short evidence document containing Challenges A–D and the failed constraint test.

SUBMISSION STANDARD

A fact table that loads but does not reconcile to the source is not considered complete. Your evidence must prove both referential integrity and numerical reconciliation.